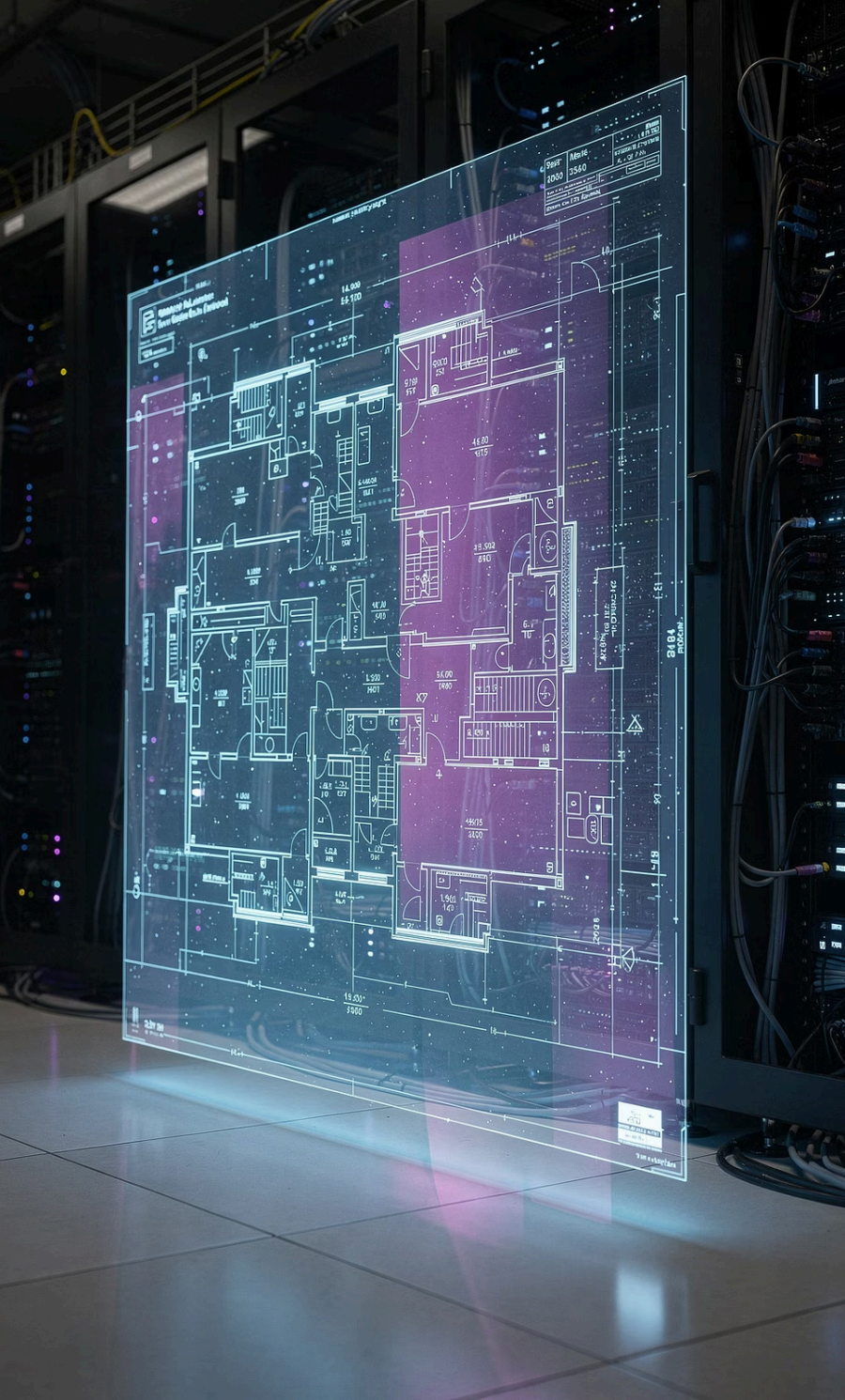




Session Persistence & Memory

Production agent design for ADK.

ADK PRODUCTION AGENTS

SESSION & MEMORY DESIGN

What is the core idea?

- **Sessions** keep a conversation continuous so the agent can respond in context from one turn to the next.
- **State** holds the current working context, such as the task in progress, active assumptions, and any intermediate results.
- **Memory** stores durable user knowledge that should survive beyond a single session when it is appropriate to retain it.
- **History** records what happened before, giving the agent a trace of prior actions, decisions, and outcomes.

Why does this matter?

- It helps the agent stay consistent, so follow-up responses reflect what the user already said rather than starting over each time.
- It improves user experience by reducing repetition, especially in longer workflows where the same context is reused across multiple turns.
- It gives production systems a clearer separation between temporary context and long-term knowledge, which makes behavior easier to reason about.

What Should You Already Understand?

Quick baseline check before we begin. If any of these feel fuzzy, this session will clarify each one.

1

Do you know what a session is and why it has a boundary?

2

Can you explain what belongs in state vs. what should never go there?

3

Do you understand the difference between in-session memory and persistent memory?

4

Can you explain how history differs from both state and memory?

5

Do you know why Redis and a database serve different purposes?

6

Can you explain what makes agent behavior predictable in production?

In-Memory State Cannot Guarantee Continuity

What is the core problem?

- In-memory state is fast for active sessions, but it is temporary by design and disappears when the process restarts.
- Any crash, deploy, or autoscaling event can erase the user's current context, including names, selections, and workflow progress.
- Because it offers no process resilience, it cannot reliably protect the state you need to carry across requests.
- For production systems, it should only support persistent session storage, not replace it.

 A restart can erase the customer name, last product viewed, or active workflow step.

What is the practical consequence?

- When state disappears unexpectedly, users can lose the customer name, the active workflow step, or the current intent they were in the middle of expressing.
- That loss makes the system feel inconsistent or forgetful, which quickly breaks user trust.
- The correct mental model is that in-memory state can support a durable, persistent session strategy, but it does not replace one.
- In production, continuity must come from storage that survives restarts and failures.

 Never use in-memory state as production persistence.

A Session Is the Conversation Room

In ADK, a **session** is the bounded thread that keeps one user's active exchange organized while the conversation is in progress. Think of it as a room that stays organized while the conversation is open: useful context stays on the table, but it is not meant to become the permanent archive.

What is a session?

- It is the bounded thread for one user’s current exchange, kept separate from other users.
- It holds the organized context needed to continue the active conversation without mixing threads.
- It is designed for continuity during the live interaction, not for long-term storage after the conversation ends.

What is the boundary rule?

- If it only matters while the room is open, keep it in the session.
- If it must survive after the room closes, move it to persistent storage.
- Use the session for live interaction state, and use storage for anything that needs durability beyond the exchange.

 Session state is temporary by design; persistent storage is for anything that must survive after the session ends.

Session Lifecycle

- It starts when the user begins an active exchange.
- It ends when the conversation closes, times out, or is handed off.
- When it closes, session state is no longer the right place for the data to live.

What Belongs Inside

- Current intent, so the assistant knows what the user is trying to do right now.
- Selected option, so follow-up messages stay aligned with the same choice.
- Retry count, workflow step, and temporary flags, so the system can continue smoothly within the active exchange.

What Stays Out

- Archival records and cross-user data, because they do not belong to one active conversation thread.
- Durable facts and source-of-truth records, because they must survive beyond the session.
- Anything that must remain available after the session ends, because that belongs in persistent storage.

Memory Is the Whiteboard in the Room

What `session.state` does

- It is the agent’s scratchpad for the current conversation, keeping the immediate working context available across turns.
- It holds dynamic values like current intent, selected options, and intermediate results that matter right now.
- It lives inside the session and is scoped to the current exchange, so it changes as the conversation evolves.
- It gives the agent a structured place to remember what it has learned during the active interaction.

`session.state` is the implementation that carries the temporary context the agent needs while the conversation is active.

How to use it correctly

- Read and write through **`session.state`**, not through local variables, so the state stays part of the framework’s managed flow.
- Let updates move through the session and event system, so changes remain traceable and debuggable across turns.
- Avoid mutating state outside that flow, because the framework will not record the change and the agent can appear to forget.
- Keep state organized and consistent, rather than scattering it across ad hoc objects that are hard to follow.

❏ **When should you use it?** Use session memory for anything that should influence the current conversation. If it must survive after the session ends, move it to persistent storage instead.

Scratchpad, Not Archive Use it for what matters now, not for long-term retention.	Traceable Updates Changes flow through session events, making them debuggable.	Structured, Not Ad Hoc Avoid scattering state in local objects; use <code>session.state</code> consistently.
---	--	---

The Notebook That Stays With the User

What is persistent memory?

- It survives beyond a single session, so the app can remember useful context when the user returns later.
- It typically has two layers: working memory for active context and long-term memory for durable, reusable facts.
- It reuses context across visits without requiring the agent to relearn the same details every time.
- It is meant to store user-relevant state, not to change the agent's core logic or behavior.

Session ends. The **notebook goes with the user**, so the experience can continue where it left off.

When should you use it?

→ Why store preferences and settings?

Because durable choices like language, formatting, or defaults should carry forward across sessions.

→ Why keep repeat customer signals?

Because recognizing returning users lets the system personalize responses and reduce repeated questions.

→ What belongs in saved context?

Profile facts and other stable details that help the user pick up quickly without reintroducing themselves.

 **What should you avoid?** Highly transient details, because persistent memory should stay stable, relevant, and valuable over time.

What Is History, and How Is It Different from Memory?

History is the full transcript of the conversation. It records what happened, but it is not the same as working memory or selective context.

What Is History?

- History is the complete session transcript, including everything the user said and everything the agent replied.
- It also includes tool actions and other interaction events, so it can be used to replay the session accurately.
- Use history when you need a faithful record of the conversation, not when you need a compact decision-making context.
- It grows over time, so it is useful for audit, debugging, and review.

What Is Memory?

- Memory is selective context, chosen because it helps the agent make better decisions right now.
- It should preserve only the information that stays relevant across turns, rather than every past message.
- Good memory stays focused, stable, and easy to reuse without overwhelming the prompt.
- It improves current actions by surfacing the details that matter most for the task at hand.

Why Does This Matter?

- If you treat history like memory, the transcript can become too large and create unnecessary context bloat.
- If you keep memory too broad, it can distract from the current task instead of improving it.
- The right balance helps the system stay accurate, efficient, and responsive as conversations get longer.
- In practice, history supports replay and analysis, while memory supports better live behavior.

Session State, Events, and Service

1

What is Session State?

- Session state is conversation-specific data that changes as the interaction progresses.
- Read and write it through `session.state` so the runtime can manage it consistently.
- Update it through the flow instead of mutating local variables, which keeps behavior traceable.
- Use it for information the agent needs to remember right now, not for temporary scratchpad state.
- Because it is managed centrally, the state stays consistent across turns and components.

2

What are session.events?

- `session.events` is the chronological log of what happened during the session.
- It captures user messages, model outputs, and tool actions in the order they occurred.
- This makes it the primary source for traceability when you need to understand a decision.
- Use it for debugging, replay, and diagnosing why the agent produced a specific response.
- Events explain the history of behavior, even when the current state only shows the final result.

3

What does the Session Service do?

- The session service binds state to the right user and application context.
- It stores session data durably so important context survives beyond a single in-memory process.
- Use it when the runtime needs reliable persistence rather than volatile memory.
- It is part of the runtime contract, which means state handling should follow its rules and expectations.
- In practice, it keeps both state and events dependable enough for production use.

📌 What should you remember? Events tell you what happened, state tells you what matters now, and the session service keeps both reliable and tied to the right context.

Right Tool, Right Job

What is Redis for?

Fast session continuity when you need working memory that is quick to read and write.

- Use it for short-lived context that helps the agent continue a conversation without delay.
- Use it when retrieval speed matters more than long-term archival.
- Use it for search or caching layers where you want fast access and simple updates.
- Do not put business logic in Redis; keep it focused on state that needs to move quickly.

✓ **Use Redis when the main goal is to remember the conversation quickly and reliably.**

What is the database for?

Durable, long-lived records that need to survive over time and support auditability.

- Use it for archival memory, session history, and records you may need to inspect later.
- Use it for business data that must remain consistent and durable across restarts.
- Use it when the information matters beyond the current interaction and should not be treated as temporary.
- Choose it when storage reliability is more important than instant retrieval.

📄 Practical rule:

Redis = working memory

Database = durable memory

session.events = transcript of what happened

Context, State, Memory, History

These four concepts are related but distinct. Mixing them up is one of the most common sources of reliability bugs in production agents.

Concept	What it is	Scope	Lives in	Use it for
Context Window	What the model sees in one call	Single model call	Prompt / input	Immediate reasoning and extraction
Session State	Working data for the active conversation	One session	ADK session.state	Task progress, selections, in-flight decisions
Persistent Memory	Durable facts that survive sessions	User or app lifetime	Redis or database	Preferences, profile facts, cross-session recall
Conversation History	Full transcript of what happened	Whole session	session.events / logs	Audit, replay, debugging

Choose the Right Storage

Match the data to the job: short-term, durable, or auditable.

You need to remember something for this conversation only

- Use **session.state** for the current intent, the selected product, the retry count, or the active workflow step.
- Keep it for values that should carry forward within this conversation but disappear when the session ends.
- Read and write it quickly while the agent is working through the task.
- Let ADK manage it as the session's short-term working state.

It resets when the session ends, so it is ideal for fast, in-flight coordination.

You need to remember something across future sessions

- Use **persistent memory** in Redis or a database for user preferences, language settings, profile facts, or returning-user signals.
- Store facts that should survive restarts and reload automatically in a new session.
- Keep only stable, meaningful information that will still be useful later.
- Avoid storing every detail of the conversation here.

It survives between sessions, so it is the right place for durable recall.

You need a faithful record of what happened

- Use **conversation history** in **session.events** for debugging a bad response, compliance audit, replaying a session, or understanding agent decisions.
- Keep it as the exact transcript and event trail of what occurred.
- Use it for inspection and accountability, not as a scratchpad.
- Rely on it as the source of truth for what happened.

It is history, not working memory — perfect for traceability, not active behavior.

Test Session Behavior Like Production

What should you test?



Can state recover after a restart?

Verify Redis- or database-backed state comes back exactly as expected after the process restarts, so users do not lose in-flight work.



Does state stay bound to the right session?

Make sure data never leaks across users or conversations, especially when sessions overlap or IDs are reused.



Is history separate from working state?

Confirm the transcript remains accurate even when temporary session state is cleared, expired, or rebuilt from storage.



Does persistent memory reload cleanly?

Check that remembered preferences or profile facts reappear in a new session without pulling in stale workflow state.

What must you verify?

- After a process restart, because in-memory assumptions often fail once the service comes back up.
- After a Redis or database restore, to confirm durable state is still readable and consistent.
- At session expiry, so cleanup does not accidentally erase data that should persist elsewhere.
- Across unrelated sessions, which helps catch cross-user leakage and broken session scoping.
- When state changes, to ensure updates land in the right store and are reflected consistently.

 Why does this matter? Memory design is a reliability problem, not just a prompt problem.

What Goes Wrong — and What to Do

What are the failure modes?

- **Missing session IDs:** the system cannot reliably attach state to the right user or conversation.
- **Stale or partial state:** some data survives a restart, but not enough to reconstruct the full context.
- **Schema mismatches:** old stored state no longer matches the current code or data shape.
- **Redis outages:** the storage layer disappears temporarily, so reads and writes fail or time out.
- **Diverging history:** the transcript and working memory drift apart, creating conflicting sources of truth.

How should recovery behave?

- **Fail safely:** stop short of making confident claims when the session state is incomplete or untrusted.
- **Recover only trusted data:** reload what can be verified, and leave ambiguous state out of the session.
- **Use durable storage:** keep the state you care about in a system that survives restarts and outages.
- **Degrade to transient mode:** continue the interaction without long-term memory if persistence is unavailable.
- **Explain context loss:** tell the user what happened so the behavior feels predictable instead of broken.

What should never happen?

- **Confuse transcript with memory:** conversation history is not the same thing as durable working state.
- **Trust one turn forever:** a single message should not become permanent truth without validation.
- **Store runtime objects:** in-memory objects and live pointers should not be treated as persistent records.
- **Mutate state outside flow:** updates need a controlled path so they remain consistent and observable.
- **Assume persistence exists:** design for the case where storage is missing, delayed, or partially restored.

⊗ **Why does this matter?** No persistence means no continuity. Storage and durability are architecture decisions, not implementation details.